

LiV-GS: LiDAR-Vision Integration for 3D Gaussian Splatting SLAM in Outdoor Environments

Renxiang Xiao , Wei Liu, Yushuai Chen, and Liang Hu , *Senior Member, IEEE*

Abstract—We present LiV-GS, a LiDAR-visual SLAM system in outdoor environments that leverages 3D Gaussian as a differentiable spatial representation. Notably, LiV-GS is the first method that directly aligns discrete and sparse LiDAR data with continuous differentiable Gaussian maps in large-scale outdoor scenes, overcoming the limitation of fixed resolution in traditional LiDAR mapping. The system aligns point clouds with Gaussian maps using shared covariance attributes for front-end tracking and integrates the normal orientation into the loss function to refine the Gaussian map. To reliably and stably update Gaussians outside the LiDAR field of view, we introduce a novel conditional Gaussian constraint that aligns these Gaussians closely with the nearest reliable ones. The targeted adjustment enables LiV-GS to achieve fast and accurate mapping with novel view synthesis at a rate of 7.98 FPS. Extensive comparative experiments demonstrate LiV-GS's superior performance in SLAM, image rendering and mapping. The successful cross-modal radar-LiDAR localization highlights the potential of LiV-GS for applications in cross-modal semantic positioning and object segmentation with Gaussian maps.

Index Terms—Mapping, SLAM, range sensing, multi-sensor fusion, 3D Gaussian splatting.

I. INTRODUCTION

SLAM (Simultaneous Localization and Mapping) is essential for large-scale scene reconstruction which rebuilds observed scenes based on precise positioning. As two commonly used sensors for scene reconstruction, LiDAR and cameras have shaped the development of SLAM technology. Traditional LiDAR SLAM exploits geometric and precise depth information in point clouds for accurate positioning. The recent advent of neural radiance fields (NeRFs) [1] and 3D Gaussian Splatting (3DGS) [2] has enriched visual SLAM with more detailed environmental representations. Traditional map representations such as point clouds, voxels, and surfels suffer from bounded-resolution in mapping [3], while implicit neural fields demand expensive computational resources. In contrast, 3DGS uses Gaussian ellipsoids as a highly effective map, allowing

for adaptive spatial feature representation and fast rendering. Considering the needs of accurate SLAM and photo-realistic scene reconstruction, 3D Gaussian splatting emerges as a powerful approach that effectively combines the advantages of both LiDAR and camera sensors.

Outdoor SLAM and scene reconstruction face unique challenges, such as lighting variations and unbounded depth scales, which make indoor RGBD-based solutions inadequate [4], [5], [6], [7], [8]. While LiDAR excels in precise distance measurement in outdoor environments, maintaining pixel correspondence across different views is challenging due to the sparse and discontinuous nature of LiDAR point clouds. Existing methods such as handheld mapping, rely on repetitive mapping to enhance image depth, which is impractical for mobile robot platforms that capture only a single view from the camera's perspective.

To the end, we introduce LiV-GS, a SLAM framework that uses 3D Gaussian spatial representations to seamlessly integrate LiDAR and camera images. Our method estimates robot pose by aligning Gaussian covariance from rendering with the current observations, with the back-end correcting drift and updating the Gaussian map. To overcome depth continuity issues between vision and LiDAR in unbounded scenes, we propose a Gaussian splitting method based on LiDAR point clouds, ensuring proper distribution constraints in map updates.

Our research contributions are summarized as follows:

- 1) We propose a unified LiDAR-camera outdoor SLAM framework using 3D Gaussian representations, enabling incremental mapping and high-quality new view synthesis during the process of high-precision positioning;
- 2) We introduce effective Gaussian-LiDAR alignment methods, including a normal direction constraint for stable tracking and a density- and normal-consistency-based weighting mechanism to account for the reliability of different Gaussians;
- 3) We propose a conditional Gaussian distribution constraint for map updates, allowing the propagation of reliable Gaussians with LiDAR priors to represent the entire scene, even including objects or segments where LiDAR points are unavailable.

II. RELATED WORKS

A. Large Scale Reconstruction

Existing tasks for large-scale scene reconstruction rely on known poses and sparse point clouds to render environments. For unbounded large scenes, major improvements have primarily focused on constraining Gaussian map from a Bird's Eye View (BEV) perspective, leveraging Levels of Detail (LoD), or partitioning rendering regions to accelerate rendering and enhance computational stability. Extending on the original work

Received 17 August 2024; accepted 18 November 2024. Date of publication 25 November 2024; date of current version 5 December 2024. This article was recommended for publication by Associate Editor X. Zuo and Editor J. Civera upon evaluation of the reviewers' comments. This work was supported in part by the Science Center Program of National Natural Science Foundation of China under Grant 62188101, and in part by the Shenzhen Science and Technology Program under Grant JCYJ20220818103000001. (Renxiang Xiao and Wei Liu are contributed equally to this work.) (Corresponding author: Liang Hu.)

The authors are with the Department of Automation, School of Mechanical Engineering and Automation, Harbin Institute of Technology, Shenzhen 150001, China (e-mail: l.hu@hit.edu.cn).

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2024.3505777>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2024.3505777

TABLE I
COMPARISON OF 3DGS-BASED LiDAR-VISUAL FUSION METHODS

Methods	Task	Pose acquisition	Region of Scene Reconstruction
DrivingGaussian [15]	Reconstruction	Colmap	Within FOV of Camera and LiDAR range
LIV-GaussianMap [18]	Reconstruction	LiDAR-inertial SLAM	Within FOV of Camera and LiDAR range
MMGaussian [21]	Reconstruction + SLAM	Point cloud Match Prior	Within FOV of Camera and LiDAR range
Our LiV-GS	Reconstruction + SLAM	LiDAR Point clouds and Gaussian map alignment	Within FOV of Camera

of 3D Gaussian Splatting, [9] proposes a tree-like structure that uses the projected size within a view to selectively include or exclude specific nodes, enhancing rendering efficiency. The research work DoGaussian [10] introduces improvements through partitioned rendering techniques. GaussianCity in [11] innovatively handles point clouds by transforming the visual perspective into BEV and employing an encode-decode strategy for efficient processing. Octree-GS [12] utilizes the octree data structure to effectively manage multi-level details of a scene. EfficientGS [13] incorporates a selective densification strategy and applies sparse order increments in spherical harmonics (SH). GaussianPro [14] introduces a progressive propagation approach for Gaussians, using depth and normal constraints to render large textureless scene areas effectively.

B. 3DGS-Based LiDAR-Visual Fusion

The integration of LiDAR data significantly enhances the capabilities of 3DGS in large-scale scene reconstruction. The utilization of LiDAR involves using point clouds maps and poses obtained from LiDAR SLAM as initial inputs, replacing traditional SfM data. For instance, DrivingGaussian [15] extracts image features and projects merged LiDAR scan frames, enhancing feature integration. 3DGS-ReLoc [16] and Gaussian-LIC [17] train 3D maps using direct inputs from LiDAR point clouds and corresponding images, achieving more detailed environmental models. LIV-GaussianMap [18] advances point clouds management by employing an explicit octree structure, while TCLC-GS [19] builds a hierarchical octree of implicit feature grids, using color projection for further 3DGS optimization. LetsGo [20] integrates Level of Detail (LoD) rendering into 3DGS, using Gaussian functions at various resolutions to represent 3D scenes efficiently.

Another approach integrates 3DGS directly into the SLAM process. For example, MM-Gaussian [21] uses a point cloud registration algorithm to estimate camera pose and directly merges these point clouds into the map for enhanced optimization. This method enables visualization of incremental mapping and leverages high-fidelity scene reconstruction to reverse-optimize pose, achieving near-optimal positioning accuracy and mapping quality simultaneously. These methodologies collectively highlight the pivotal role of LiDAR in refining 3D reconstruction processes.

The most closely-related works to our research include DrivingGaussian [15], LIV-GaussMap [18], and MM-Gaussian [21]. As summarized in Table I, the existing methods typically separate the tasks of SLAM and dense Gaussian mapping, where either LiDAR point clouds are used to replace SfM or point clouds are directly matched as tracking priors, and then the pose is optimized with a rendering loss based on the Gaussian map. In contrast, our LiV-GS framework optimizes both pose estimation and map updates within an integrated Gaussian map representation. Moreover, while existing 3DGS-based outdoor SLAM

methods focus solely on regions where both LiDAR point clouds and visual pixels are captured, our approach reconstructs scenes beyond the LiDAR field of view with high quality, thanks to our novel Gaussian splitting method derived from LiDAR-based Gaussians.

III. METHODOLOGY

Our LiV-GS is an outdoor visual-LiDAR SLAM system that employs 3D gaussian for environmental representation. The Gaussian distribution is mathematically expressed as:

$$G(\mathbf{x}) = e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})} \quad (1)$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ denote the mean and covariance matrix of the Gaussians, respectively. In our model, each Gaussian is defined by $g_i = \{\alpha, c, \boldsymbol{\mu}, \boldsymbol{\Sigma}\}$, where α signifies opacity, and c the color, directly derived from the original pixel data. The same as [22], we omit the spherical harmonics for simplicity and speed.

A. System Overview

As depicted in Fig. 1, the entire system of LiV-GS comprises four modules: Data Preprocessing, Tracking, Optimization and Map Update. The multi-modal measurements from LiDAR and visual sensors are integrated in Data Preprocessing and then fed into the front-end Tracking module. In the front-end, the system employs Gaussian ellipsoids rendered from the previous time instant in conjunction with the current point clouds for frame-to-map matching. Keyframes are evaluated through the co-visibility in visual regions and are appended to the Gaussian map. The Gaussian map incorporating keyframe parameters is then processed in the back-end for pose optimization and map updates. By minimizing the loss function, the Gaussian map updates the parameters of Gaussians continuously together with splitting and pruning operations of Gaussians.

In the proposed system, data inputs consist of imagery from a camera and point clouds from a LiDAR sensor. These inputs are integrated using calibrated extrinsic to transform the time-aligned LiDAR point clouds into depth images.

The transformation is governed by the equation:

$$\text{Depth} = \mathbf{K} \cdot (\mathbf{R}_L^C \cdot \mathbf{P} + \mathbf{t}_L^C) \quad (2)$$

where $\mathbf{P}$ represents the points in the LiDAR point clouds, $\mathbf{R}_L^C$ and $\mathbf{t}_L^C$ denote the rotation matrix and translation vector from the LiDAR to the camera coordinate system, respectively, and $\mathbf{K}$ is the intrinsic matrix of the camera. This transformation aligns the LiDAR data with the visual data, facilitating a unified depth perception from the RGB viewpoint.

B. Front-End Tracking

The primary challenge in front-end tracking arises from how to align variably dense Gaussians derived from color supervision with LiDAR point clouds that possess a fixed resolution. Inspired by [4], [23] and [24], we adopt covariance, a shared attribute of point clouds and Gaussians, as a bridge for integration. After determining the normal orientation for each LiDAR point, we align it with the shortest axis of the Gaussians. To further facilitate stable tracking, we introduce a weighting function for Gaussians that distinguishes Gaussians generated solely by color supervision and those also by LiDAR depth in the error calculation of point clouds and Gaussian match.

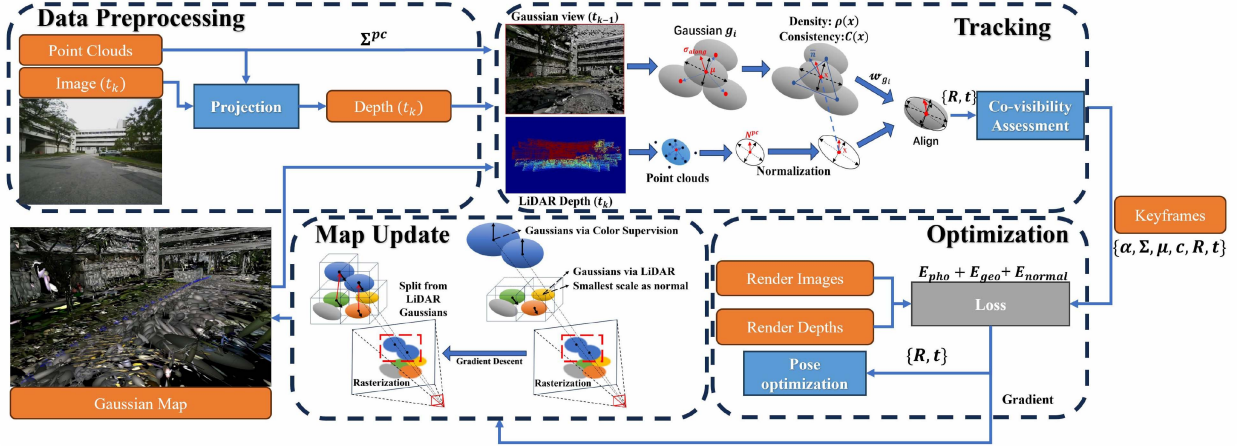


Fig. 1. **Overview of the system:** The SLAM system comprises a tracking and optimization process that together support the visual representation of the Gaussian map. The map update process uses LiDAR depth and color supervision to adjust the new Gaussians.

Point clouds and Gaussians match: Initially, we maintain a sliding window that filters and selects Gaussians from the most recent 10-time frames while masking out the remaining Gaussians. This selection process keeps Gaussians relevant for matching within our focused sub-maps. We then utilize an incremental error minimization function to ensure precise correspondences between planes and points as below:

$$E(\Sigma, \mathbf{x}_g) = \sum_{\Sigma \in P_L} w_{x_g} \left(\mathbf{n}_{x_p} \cdot \left(\mathbf{T}_W^{C_{t-1}(k)} \mathbf{x}_p - \mathbf{x}_g \right)^2 \right) + R \quad (3)$$

where $\mathbf{x}_p$ represents a point in the LiDAR point clouds P_L , $\mathbf{T}_W^{C_{t-1}(k)}$ corresponds to the current pose estimation after k^{th} iterations based on the pose from the previous moment to the world coordinate system and $\mathbf{x}_g$ denotes the center of the Gaussian closest to $\mathbf{T}_W^{C_{t-1}(k)} \mathbf{x}_p$, and $\mathbf{n}_{x_p}$ is the normal vector of $\mathbf{x}_p$. w_{x_g} is a weight representing the confidence of the point $\mathbf{x}_g$, which is detailed in the next paragraph.

The regularization term R is introduced to further enhance the stability and precision of error function, which considers the directional error between normals:

$$R = \|1 - \mathbf{n}_{x_p} \cdot \mathbf{n}_{x_g}\| \quad (4)$$

The regularization term would be multiplied by a hyperparameter for adjustment, aims to reinforce the alignment of normals.

Weighting function construction: The reliable weight of the point clouds is closely linked to its density, as depicted in Fig. 2. To construct a weight function that combines normal consistency and density factors, we first identify all the nearest Gaussian distribution centers inside the local spherical region $N_r(\mathbf{x})$, where $\mathbf{x}$ is the center of the sphere and r is the radius. These Gaussian centers are determined by minimizing the distance from point $\mathbf{x}$ to each Gaussian center μ_{g_i} . The density function $\rho(\mathbf{x})$ is given below:

$$\rho(\mathbf{x}) = \sum_{g_i \in N_r(\mathbf{x})} \alpha_i \exp \left(-\frac{1}{2} (\mathbf{x} - \mu_{g_i})^T \Sigma_{g_i}'^{-1} (\mathbf{x} - \mu_{g_i}) \right) \quad (5)$$

$$\Sigma_{g_i}' = D_{g_i} \text{diag}(1, \Sigma_{\text{perp1}}/\Sigma_{\text{along}}, \Sigma_{\text{perp2}}/\Sigma_{\text{along}}) D_{g_i}^T \quad (6)$$

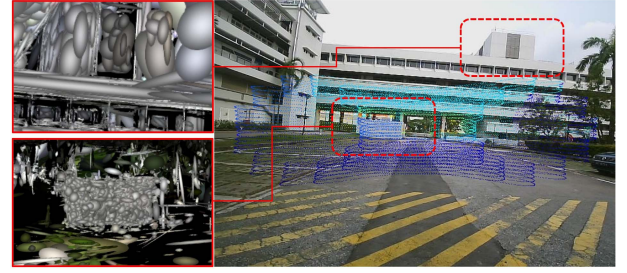


Fig. 2. **Relationship between Density and weight:** Gaussians based on only color supervision result in isotropic and sparse Gaussians (top left). Regions with dense depth input from LiDAR typically show Gaussians in higher density (bottom left).

where Σ_{g_i}' is the reconstructed covariance matrix, achieved by selecting the smallest variance Σ_{along} along the normal direction and a larger variance Σ_{perp} in the perpendicular directions. D_{g_i} is an orthogonal basis formed by the normal and its perpendiculars. Since the length of the Gaussian normal is difficult to restrict during the optimization, we introduced the normal length normalization for both point clouds and Gaussians, so that the robustness of the tracking algorithm can be ensured by the stable orientation of the normal.

In addition, since (5) involves calculations of the density of Gaussians per point during the matching process, to speed up the calculation process, we simplify the density function calculation in (5) while tracking:

$$(\mathbf{x} - \mu_g)^T \Sigma_g'^{-1} (\mathbf{x} - \mu_g) \approx \frac{\Sigma_{\text{perp1}} \Sigma_{\text{perp2}}}{\Sigma_{\text{along}}^2} \langle \mathbf{x} - \mu_g, \mathbf{n}_{x_g} \rangle^2 \quad (7)$$

For each point $\mathbf{x}$, we then calculate the consistency $C(\mathbf{x})$ between the normal $\mathbf{n}_{x_g}$ of the current Gaussian distribution and local average normal $\bar{\mathbf{n}}$ as $C(\mathbf{x}) = \mathbf{n}_{x_g} \cdot \bar{\mathbf{n}}$. The final weight function $W(\mathbf{x})$ is defined as the product of the normal consistency and the density function $W(\mathbf{x}) = C(\mathbf{x})\rho(\mathbf{x})$.

Co-visibility assessment: We measure co-visibility by assessing the overlap of Gaussian functions between the current and last keyframe. Should co-visibility fall below a specified threshold, the frame is designated as a keyframe. To guarantee

that fast tracking under a forward-looking perspective consistently matches the current point clouds with sufficient Gaussian ellipsoids, point clouds from the keyframes are added to the Gaussian map directly, using pixel colors and scaling point clouds covariance along the ray propagation direction for initialization. Subsequently, the back-end optimizes the augmented Gaussian map by comparing it against the perspectives of previous keyframes, leading to incremental updates of the Gaussian map.

C. Back-End Optimization

The back-end optimization process retrieves a sequence of keyframe identifiers along with their corresponding parameters and conducts two rounds of optimization. The first time is confined to the poses of keyframes within the sliding window, while the second time is aimed at updating the Gaussian map. The depth and color rendering process from the 3D Gaussian map G^s is derived as:

$$D_{\text{render}} = \sum_{g_i \in G^s} d_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (8)$$

$$C_{\text{render}} = \sum_{g_i \in G^s} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (9)$$

where d_i and c_i represent the distance and color to the Gaussian g_i along the camera ray.

The loss function used for optimizing the parameters of Gaussians is designed as:

$$\mathcal{L} = (1 - \lambda_1)E_{\text{pho}} + \lambda_1 E_{\text{geo}} + \lambda_2 E_{\text{normal}} \quad (10)$$

where the first two terms are commonly used in existing research works: the photometric error E_{pho} represents the discrepancy between the visual ground truth and the rendered image, while the geometric error E_{geo} measures the disparity between the LiDAR depth input and the rendered depth image. The third term $E_{\text{normal}} = \|\Sigma_{\text{along}}\|$ quantifies the norm of the average variance along the normal direction.

Unlike indoor 3D Gaussian splatting where stacked Gaussians are constructed from dense RGB-D images, outdoor scenes typically feature larger and sparser Gaussians. To align Gaussians with object surfaces closely, we introduce the normal loss E_{normal} which is optimized for shorter and more stable normals. As demonstrated in Fig. 3, with the introduced normal loss, the 3D Gaussian map are structured with more stable normals, improving rendering quality greatly.

D. Map Update

To manage the representation of the sky in unbounded scenes, we adopt a method similar to [9], initializing a skybox outside the scene with 100,000 Gaussian primitives. This skybox is dynamically updated as the Gaussian map is incrementally built. During map initialization and update, two types of Gaussians are predominant: Gaussians derived from color supervision X and those provided by LiDAR measurements Y .

We introduce a Conditional Gaussian Constraint (CGC) to adjust the positions of color-supervised Gaussians through the loss function (10). For each point x acquired through color supervision, the nearest Gaussian y is selected from the LiDAR-measured Gaussians set. Further, it is postulated that given y , x

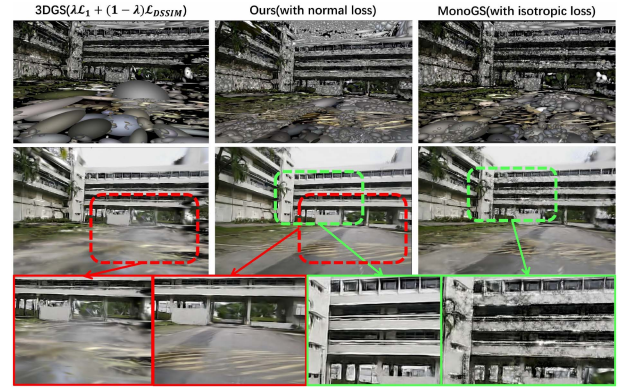


Fig. 3. **Effect of Normal Restriction:** **Top:** Ellipsoid visualization. **Middle:** Render images. **Bottom:** Magnified details of the render. The left comparison (in red) illustrates uncontrolled Gaussian growth leading to significant artifacts. The right comparison (in green) shows gaps in the rendered images caused by isotropic Gaussians. Our method effectively prevents these issues.

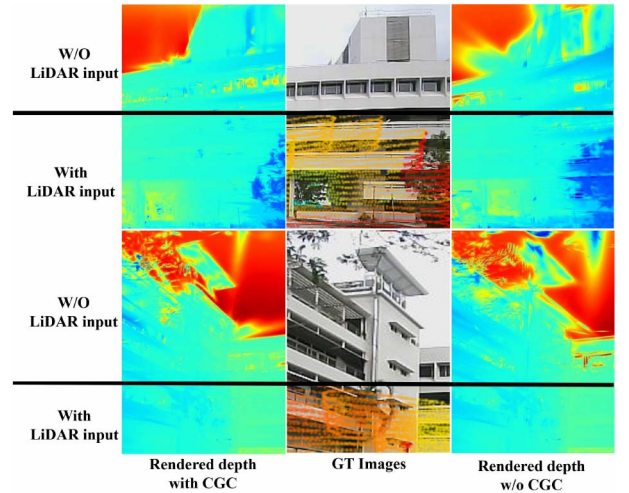


Fig. 4. **Effect of Splitting via conditional Gaussian constraints (CGC).** Our approach enhances the representation of Gaussians for objects in the images that lack LiDAR depth input via the introduced CGC.

follows a Gaussian distribution:

$$p(X|Y = y) \sim \mathcal{N}(\mu_x(y), \Sigma_y) \quad (11)$$

where $\mu_x(y)$ is the location of new Gaussian ellipsoid split from Gaussian y through normal distribution sampling, and Σ_y is the covariance of Gaussian y .

The conditional Gaussian (11) adjusts the mean μ_x and covariance Σ_x of x , aligning them more closely with the nearest reliable Gaussian y . The new Gaussian split from the reliable Gaussian is regarded as reliable Gaussian after undergoing a round of back-end optimization, the process of which continue until all Gaussians become reliable Gaussians. As shown in Fig. 4, the newly-split points will strictly adhere to the distribution patterns of existing reliable points, especially in areas with complex shapes or distinctive surface features.

IV. EXPERIMENT

In the experiments, we evaluate LiV-GS and compare it against other SOTA algorithms from three aspects: localization

TABLE II
QUANTITATIVE ANALYSIS FOR TRACKING ACCURACY

Methods	cp			garden1			garden2			nyl1			nyl2			loop2		
	$t_{rel} \downarrow$	$r_{rel} \downarrow$	$t_{abs} \downarrow$	$t_{rel} \downarrow$	$r_{rel} \downarrow$	$t_{abs} \downarrow$	$t_{rel} \downarrow$	$r_{rel} \downarrow$	$t_{abs} \downarrow$	$t_{rel} \downarrow$	$r_{rel} \downarrow$	$t_{abs} \downarrow$	$t_{rel} \downarrow$	$r_{rel} \downarrow$	$t_{abs} \downarrow$	$t_{rel} \downarrow$	$r_{rel} \downarrow$	$t_{abs} \downarrow$
NeRF-LOAM	2.943	9.644	5.39	1.182	0.559	0.540	1.213	0.707	1.076	1.371	1.140	3.504	1.343	1.730	17.460	1.442	2.205	1.785
HDL-graph-SLAM	1.264	1.553	1.079	1.874	1.603	1.478	1.186	0.880	3.154	1.737	1.271	2.266	1.514	1.835	17.638	1.436	2.802	0.593
ORB-SLAM3	1.356	1.992	2.865	1.173	0.626	0.529	1.212	0.772	1.001	1.342	1.172	19.528	1.333	1.736	23.283	1.403	2.256	0.952
SplatAM	-	-	2.336	-	-	0.979	-	-	1.221	-	-	12.332	-	-	17.442	-	-	2.692
MonoGS	4.171	3.472	3.440	1.179	0.754	0.664	1.163	0.765	0.708	1.382	1.175	9.595	1.371	1.701	28.553	7.375	5.708	15.357
Gaussian-SLAM	1.249	3.047	1.040	-	-	-	-	-	-	-	-	-	-	-	-	1.399	2.384	1.136
GS-ICP-SLAM	5.471	4.041	6.33	1.249	0.764	2.082	1.824	1.316	5.507	1.662	1.771	23.331	2.101	1.070	23.915	3.236	2.644	13.819
Ours	0.234	1.216	0.464	1.183	0.716	0.366	1.236	0.962	0.679	1.240	1.307	0.580	1.106	1.369	0.771	1.393	2.239	0.843

accuracy, rendering quality, and the reliability of spatial distribution of Gaussian maps.

To evaluate trajectory error, we used the open-source tool rpg trajectory evaluation [25] to compute both Absolute Trajectory Error (ATE) and Relative Error (RE), measuring the ATE root-mean-square error (RMSE) drift (m), average translational RMSE drift (%) and average rotational RMSE drift $^{\circ}/100m$.

For rendering evaluation, the optimized viewpoints from each algorithm were extracted and compared against the actual images using metrics of SSIM, PSNR[dB], and LPIPS.

A. Implementation Details

LiDAR and image data were synchronized using timestamps, and the trajectories obtained by the R³ Live [26] algorithm which integrates vision, LiDAR, and IMU data, is used as the ground truth. As the trajectory accuracy, LiV-GS is compared with the established point clouds-based geometric SLAM algorithm HDL graph SLAM [27], image feature-based visual SLAM algorithm ORB-SLAM3 [28], and implicit neural field-based NeRF-LOAM [3], and the open-source SLAM algorithms based on 3D Gaussian Splatting such as MonoGS [22], Gaussian-SLAM [29], GS-ICP-SLAM [4] and SplatAM [30]. Image rendering quality comparisons are performed under gt pose and its odometry using algorithms including 3DGS, NeRF++[31], MonoGS, Gaussian-SLAM, GS-ICP-SLAM and SplatAM. All of the algorithms were run on a desktop with an NVIDIA RTX 4090 GPU.

B. Datasets

To effectively evaluate our LiV-GS, we utilized the open-source large-scale dataset NTU4DRadLM, which includes the data collected by three different types of sensors: Livox-Horizon LiDAR at 10 Hz, a monocular camera with a resolution 640×480 , and a 4D millimeter wave radar Eagle Ocuill G7. Our method does not use the IMU data. Due to the challenge of maintaining photometric consistency in long-distance outdoor scenes, we segmented the low-speed, kilometers-long scenarios as several shorter sequences. For the cp sequence, we used the first 2400 LiDAR-camera aligned images, covering approximately 230 meters. For the garden and nyl sequences, we selected 2100 and 2400 images respectively from both the beginning and end of each sequence, with each segment covering at least 220 meters. Additionally, for the Loop2 sequence recorded on a human-driving vehicle platform, we selected 300 frames covering about 250 meters. In total, we tested six sequences to comprehensively validate our LiV-GS method.

C. Evaluation of Tracking Accuracy

Fig. 5 and Table II display qualitative and quantitative evaluations of localization accuracy, respectively. Our LiV-GS

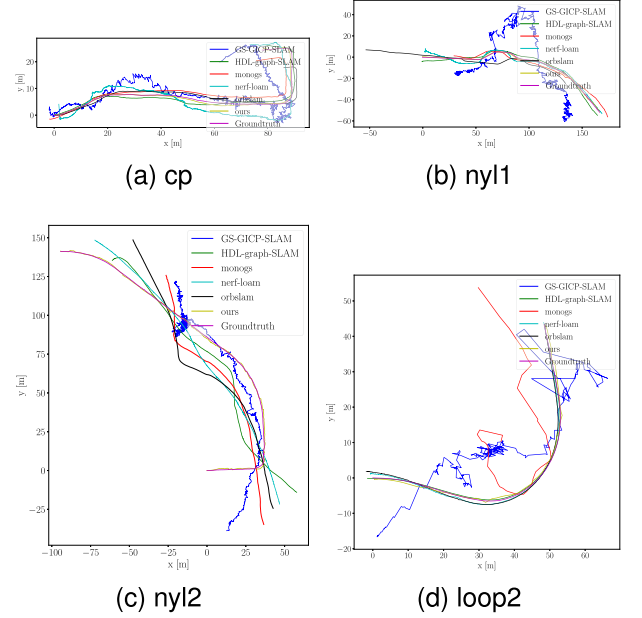


Fig. 5. Comparison of trajectories using different SLAM algorithms on four sequences of NTU4DRadLM dataset.

demonstrates the lowest ATE in low-speed sequences. However, in high-speed sequence loop2, the accuracy of LiV-GS is slightly lower than NeRF-LOAM due to larger displacement between the consecutive frames and sparse Gaussian distribution led by insufficient Gaussian map optimizations. MonoGS, SplatAM, GS-ICP-SLAM, and Gaussian-SLAM are all tailored for indoor environments with well-textured images and dense depth information, and they suffer performance degradation or even fail in some outdoor sequences due to sparser depth information obtained in outdoor scenes. In contrast, our LiV-GS tracks consistently and stably in large-scale outdoor environments.

In addition, the bottom two rows in Table III show a minimal rendering difference between the outcomes from LiV-GS odometry and ground truth, which further justifies the high precision of LiV-GS odometry, as higher localization accuracy leads to a smaller loss in rendering quality.

D. Evaluation of Rendering

Table III presents a comparison of rendering results from all reproducible open-source algorithms implemented on the same platform. 3DGS and NeRF++ do not perform pose estimation and utilize COLMAP to obtain the initial inputs. SplatAM, MonoGS and Gaussian-SLAM rely on loss functions to optimize pose, so we replicated the rendering results twice: once using

TABLE III
QUANTITATIVE ANALYSIS FOR RENDERING [SSIM↑ PSNR↑ LPIPS↓]

Methods	cp			garden1			garden2			nyl1			nyl2			loop2		
	SSIM	PSNR	LPIPS	SSIM	PSNR	LPIPS	SSIM	PSNR	LPIPS	SSIM	PSNR	LPIPS	SSIM	PSNR	LPIPS	SSIM	PSNR	LPIPS
3DGS	0.718	21.95	0.617	0.598	20.495	0.557	0.662	20.402	0.536	0.726	20.224	0.575	0.571	18.040	0.657	0.594	17.794	0.620
GS-ICP-SLAM	0.552	17.336	0.772	0.511	15.782	0.661	0.417	13.946	0.606	-	-	-	0.374	11.533	0.882	0.492	12.335	0.771
NeRF++	0.566	19.226	0.698	0.529	15.772	0.685	0.475	13.736	0.675	0.574	13.662	0.726	0.320	10.130	0.879	-	-	-
SplaTAM (Odom)	0.629	18.772	0.585	0.515	18.788	0.545	0.540	18.402	0.569	0.688	19.770	0.574	0.480	17.212	0.644	0.475	13.736	0.675
SplaTAM (GT)	0.535	17.501	0.691	0.498	17.936	0.607	0.497	18.402	0.533	0.597	18.960	0.520	0.432	16.933	0.721	0.438	11.225	0.753
MonoGS (Odom)	0.655	20.962	0.444	0.607	23.124	0.474	0.649	20.627	0.535	0.813	22.800	0.373	0.560	17.812	0.665	0.578	17.993	0.503
MonoGS (GT)	0.733	22.118	0.364	0.717	23.723	0.522	0.598	20.112	0.552	0.748	21.969	0.415	0.511	17.006	0.738	0.505	17.567	0.616
Gaussian-SLAM (Odom)	0.665	20.924	0.595	-	-	-	-	-	-	-	-	-	-	-	-	0.534	16.509	0.656
Gaussian-SLAM (GT)	0.647	21.026	0.688	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Ours (Odom)	0.775	22.274	0.336	0.773	23.569	0.386	0.772	22.141	0.336	0.833	22.559	0.274	0.725	20.835	0.559	0.686	18.037	0.556
Ours (GT)	<u>0.763</u>	<u>22.368</u>	<u>0.319</u>	<u>0.721</u>	<u>23.353</u>	<u>0.404</u>	<u>0.758</u>	<u>21.991</u>	<u>0.348</u>	0.799	22.136	<u>0.308</u>	0.744	<u>20.630</u>	0.548	<u>0.631</u>	18.472	0.417

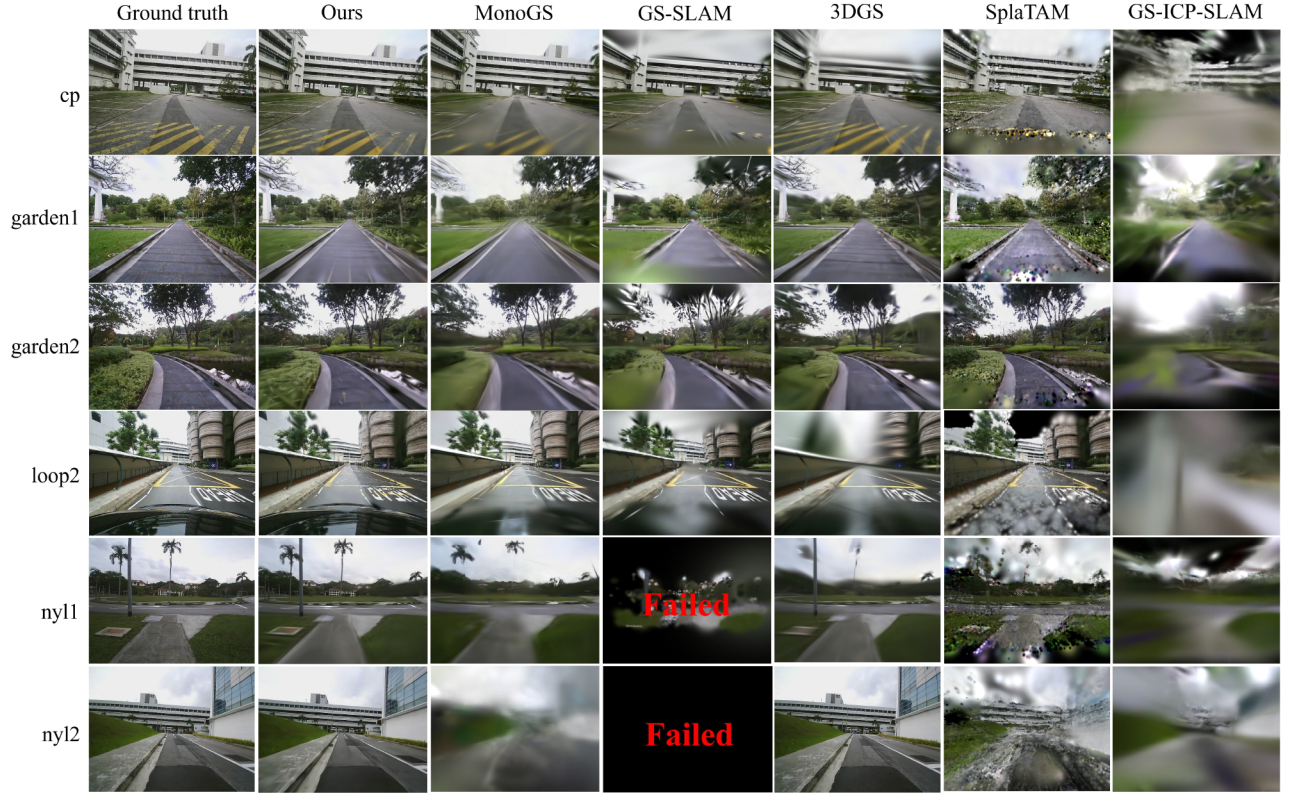


Fig. 6. Comparison of Rendering Results.

the ground truth poses and once using the poses estimated by themselves.

Fig. 6 presents the qualitative results of image rendering, the rendering quality of 3DGS is affected by the lack of depth priors and the limited perspectives obtained during the SLAM motion process. The images rendered by LiV-GS exhibit remarkable clarity, capturing nuanced details such as the architectural structure and outlines of buildings, ground textures, foliage details of trees and shrubs, and even reflections on car hoods. The fidelity in rendering not only demonstrates the effectiveness of LiV-GS but also highlights its capability in preserving intricate visual details.

E. Evaluation Between Performance and Runtime

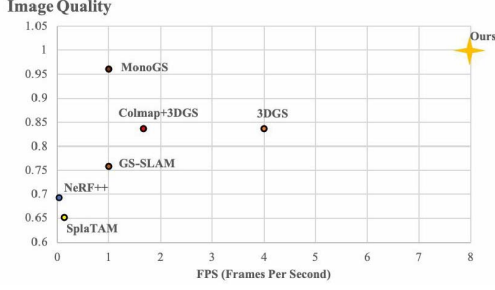
To further estimate the tradeoff between efficiency and performance of LiV-GS, we compared the relationship between runtime and previously evaluated performance metrics. The ATE RMSE is directly used to measure localization accuracy and the

image quality of each algorithm is calculated by normalizing the composite score:

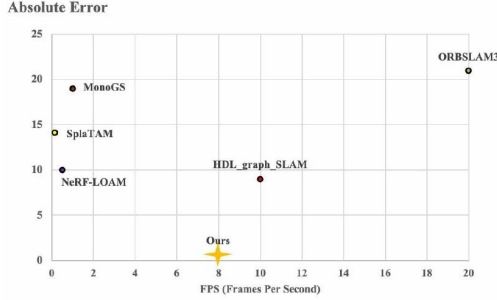
$$\text{Image Quality} = \text{SSIM} + \text{PSNR}/30 + (1 - \text{LPIPS})$$

As shown in Fig. 7(a) and 7(b), LiV-GS simultaneously reaches state-of-the-art performance in both accuracy and rendering quality with a running speed of 7.98 FPS, showing its potential for real-time SLAM application as the LiDAR sampling rate is 10 Hz.

As to the running time of each module, the front-end tracking and keyframe selection module operates at 0.07ms per operation on average. With the back-end configured with five-keyframe optimization at a time, the average runtime of the pose optimization and map update modules are 0.04 ms and 0.09 ms, respectively. It is worth noting that our LiV-GS system employs an asynchronous communication mechanism, so the operating time of the entire system does not equal the sum of the running time of individual modules.



(a) Comparison of image quality and running speed in cp



(b) Comparison of localization accuracy and running speed in nyl

Fig. 7. **Comparison of performance metrics and system runtime.** FPS is calculated as the ratio of the total number of processed frames to the total time. A value of 7.98 FPS means that on average 7.98 depth maps and RGB images can be processed in one second.

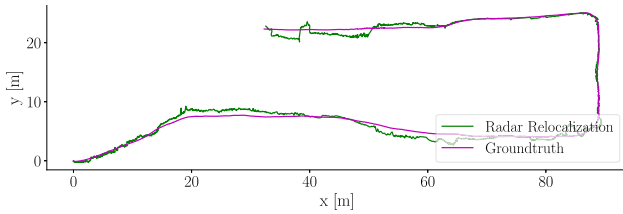


Fig. 8. **Visualization of cross-modal mmWave radar localization trajectory.**

F. Qualitative Analysis of Geometry Accuracy

To assess the spatial geometric distribution of Gaussian map constructed by LiV-GS, we conducted cross-modal mmWave radar localization on the Gaussian map. Unlike LiDAR, the point clouds of mm-Wave radar are much sparser and have lower resolution and higher noises. Robust cross-modal radar localization is possible only if the spatial geometry of the Gaussian map generated by LiDAR and images is sufficiently accurate and reliable.

For the sequence cp from NTU4DRadLM dataset, we first constructed the Gaussian map based on camera and LiDAR data using our LiV-GS, then relocalized radar data on the Gaussian map using the HDL-localization algorithm [27]. Fig. 8 highlights that even with cross-modal radar data, accurate localization is consistently achieved using Gaussian maps. It confirms that the Gaussian map of LiV-GS provides accurate geometric structure information, showing its great potential for other downstream tasks such as all-weather outdoor localization and navigation.

TABLE IV
QUANTITATIVE ANALYSIS FOR TRACKING

Methods	$t_{rel} \downarrow$	$r_{rel} \downarrow$	$t_{abs} \downarrow$
NeRF-LOAM	0.518	0.636	1.617
HDL-graph-SLAM	0.497	0.801	0.712
ORB-SLAM3	0.446	0.712	0.457
SplaTAM	-	-	-
MonoGS	0.535	0.932	1.963
Gaussian-SLAM	1.456	<u>0.562</u>	1.444
GS-ICP-SLAM	-	-	-
Ours	0.436	0.546	<u>0.705</u>

TABLE V
QUANTITATIVE ANALYSIS FOR RENDERING

Methods	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$
3DGS	0.868	25.776	0.247
GS-ICP-SLAM	0.446	11.949	0.813
NeRF++	0.581	12.256	0.791
SplaTAM	0.235	10.501	0.921
MonoGS	0.533	15.928	0.694
Gaussian-SLAM	0.399	10.523	0.928
Ours	<u>0.565</u>	<u>17.316</u>	<u>0.568</u>

G. Close-Loop Validation

In this subsection, we verify the performance of LiV-GS in closed-loop sequences. We intercepted a closed-loop sequence of the hku_park_00 data from the handheld scanning dataset provided by R³ Live [26], which includes 1280×1024 images at 30 Hz and Livox-Avia LiDAR point clouds at 10 Hz.

The quantitative results are shown in Tables IV and V. In this looped sequence, our LiV-GS still performs well but its performance falls behind some other algorithms occasionally. One reason for that is LiV-GS lacks a loop closure detection module to optimize the accumulated drift. On the other hand, Since LiV-GS does not utilize spherical harmonics to represent colors. Observing different colors of the same scene from varying viewpoints at loop closure positions could result in a substantial overlap of Gaussian ellipsoids, which might lead to performance degradation. How to address the issues of loop closure and color refinement will be the focus of our future research.

V. CONCLUSION

We propose a novel outdoor SLAM system leveraging 3D gaussian as spatial representation, integrating LiDAR-camera data. The system tightly integrates LiDAR point clouds with Gaussian maps via covariance for tracking and uses visual information to optimize the global Gaussian distribution. Conditional Gaussian constraints guide Gaussian ellipsoid splitting, especially for Gaussian ellipsoids lacking geometric constraints as LiDAR depth is unavailable. This method exploits visual texture continuity and LiDAR reliability to construct ellipsoids with consistent depth constraints.

Experiments validate that our system achieves robust, precise localization and can render clear scene imagery. To our knowledge, this is the first successful implementation of cross-modal radar-LiDAR localization using 3D Gaussian maps in outdoor environments, marking a significant potential application of outdoor 3DGS SLAM.

REFERENCES

- [1] B. Mildenhall, P.P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [3] J. Deng et al., "NeRF-LOAM: Neural implicit representation for large-scale incremental LiDAR odometry and mapping," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 8218–8227.
- [4] S. Ha, J. Yeon, and H. Yu, "RGBD gs-icp SLAM," in *Proc. European Conf. Comput. Vis.*, 2024, pp. 180–197.
- [5] Y. Tao, Y. Bhalgat, L. F. T. Fu, M. Mattamala, N. Chebrolu, and M. Fallon, "SiLVR: Scalable LiDAR-visual reconstruction with neural radiance fields for robotic inspection," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2024, pp. 17983–17989.
- [6] C. Yan et al., "GS-SLAM: Dense visual SLAM with 3D Gaussian splatting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 19595–19604.
- [7] L. C. Sun et al., "MM3DGS SLAM: Multi-modal 3d Gaussian splatting for SLAM using vision, depth, and inertial measurements," 2024, *arXiv:2404.00923*.
- [8] Q. U. Islam, H. Ibrahim, P. K. Chin, K. Lim, and M. Z. Abdullah, "MVS-SLAM: Enhanced multiview geometry for improved semantic RGBD SLAM in dynamic environment," *J. Field Robot.*, vol. 41, no. 1, pp. 109–130, 2024.
- [9] B. Kerbl et al., "A hierarchical 3D Gaussian representation for real-time rendering of very large datasets," *ACM Trans. Graph.*, vol. 43, no. 4, pp. 1–15, 2024.
- [10] Y. Chen and G. H. Lee, "DoGaussian: Distributed-oriented Gaussian splatting for large-scale 3D reconstruction via Gaussian consensus," 2024, *arXiv:2405.13943*.
- [11] H. Xie, Z. Chen, F. Hong, and Z. Liu, "GaussianCity: Generative Gaussian splatting for unbounded 3D city generation," 2024, *arXiv:2406.06526*.
- [12] K. Ren et al., "Octree-GS: Towards consistent real-time rendering with LoD-structured 3D Gaussians," 2024, *arXiv:2403.17898*.
- [13] W. Liu et al., "EfficientGS: Streamlining Gaussian Splatting for large-scale high-resolution scene representation," 2024, *arXiv:2404.12777*.
- [14] K. Cheng et al., "GaussianPro: 3D Gaussian splatting with progressive propagation," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024.
- [15] X. Zhou, Z. Lin, X. Shan, Y. Wang, D. Sun, and M.-H. Yang, "DrivingGaussian: Composite Gaussian splatting for surrounding dynamic autonomous driving scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 21634–21643.
- [16] P. Jiang, G. Pandey, and S. Saripalli, "3DGS-Reloc: 3D Gaussian splatting for map representation and visual relocalization," 2024, *arXiv:2403.11367*.
- [17] X. Lang et al., "Gaussian-LIC: Photo-realistic LiDAR-inertial-camera SLAM with 3D Gaussian Splatting," 2024, *arXiv:2404.06926*.
- [18] S. Hong, J. He, X. Zheng, C. Zheng, and S. Shen, "LiV-GaussMap: LiDAR-inertial-visual fusion for real-time 3D radiance field map rendering," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 9765–9772, Nov. 2024.
- [19] C. Zhao et al., "TCLC-GS: Tightly coupled LiDAR-camera Gaussian Splatting for surrounding autonomous driving scenes," 2024, *arXiv:2404.02410*.
- [20] J. Cui et al., "LetsGo: Large-scale garage modeling and rendering via LiDAR-assisted Gaussian primitives," *ACM Trans. Graph.*, vol. 43, no. 6, pp. 1–18, 2024.
- [21] C. Wu, Y. Duan, X. Zhang, Y. Sheng, J. Ji, and Y. Zhang, "MM-Gaussian: 3D Gaussian-based multi-modal fusion for localization and reconstruction in unbounded scenes," 2024, *arXiv:2404.04026*.
- [22] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison, "Gaussian splatting SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 18039–18048.
- [23] A. Guédon and V. Lepetit, "SuGaR: Surface-aligned Gaussian splatting for efficient 3D mesh reconstruction and high-quality mesh rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 5354–5363.
- [24] J. Behley and C. Stachniss, "Efficient surfel-based SLAM using 3D laser range data in urban environments," in *Proc. Robot.: Sci. Syst.*, vol. 2018, 2018, p. 59.
- [25] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for Visual-(Inertial) odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robot. Syst.*, 2018, pp. 7244–7251.
- [26] J. Lin and F. Zhang, "R3LIVE: A robust, real-time, RGB-colored, LiDAR-Inertial-Visual tightly-coupled state estimation and mapping package," in *Proc. 2022 IEEE Int. Conf. Robot. Automat.*, 2022, pp. 10672–10678.
- [27] K. Koide, J. Miura, and E. Menegatti, "A portable three-dimensional LiDAR-based system for long-term and wide-area people behavior measurement," *Int. J. Adv. Robotic Syst.*, vol. 16, no. 2, 2019, Art. no. 1729881419841532.
- [28] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
- [29] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-SLAM: Photo-realistic dense SLAM with Gaussian splatting," 2023, *arXiv:2312.10070*.
- [30] N. Keetha et al., "SplaTAM: Splat Track & map 3D Gaussians for Dense RGB-D SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 21357–21366.
- [31] K. Zhang, G. Riegler, N. Snavely, and V. Koltun, "NeRF: Analyzing and improving neural radiance fields," 2020, *arXiv:2010.07492*.